



Lab Title: OWASP Top 10 Risk Assessment and Secure Application Design for a Vulnerable Web Platform

Student Name: Nebeuwa Ifeanyichukwu Raphael

Student ID: 202520850LE

Course Name: BVWS102 - Web Application Security Essential

Instructor Name: Ahmed Bukar

Date of Submission: 24/11/2025

Executive Summary

This report presents a comprehensive security assessment of PayX Secure, a newly launched online payment and digital wallet platform. As part of the Web Application Security course, the objective was to evaluate the platform's exposure to the OWASP Top 10 web vulnerabilities and recommend secure design strategies to mitigate potential risks.

Through detailed analysis and hands-on testing, we identified key vulnerabilities that could be exploited by attackers, including SQL Injection, Cross-Site Scripting (XSS), and Broken Authentication. These issues pose serious threats to user data, financial transactions, and the overall trustworthiness of the platform.

To address these concerns, we mapped relevant OWASP risks to PayX Secure's architecture, conducted practical testing using a vulnerable lab environment, and proposed targeted security controls based on industry best practices. The final report outlines our findings, evidence of exploitation, and actionable recommendations to strengthen the platform's security posture.

Lab Objective

The lab component of this assignment aimed to simulate real-world attacks on vulnerable web applications to better understand how common security flaws are exploited. Using a selected lab platform (e.g., OWASP Juice Shop or DVWA), we focused on three critical vulnerabilities:

- **SQL Injection:** To demonstrate how attackers can manipulate database queries to access or alter sensitive data.
- **Cross-Site Scripting (XSS):** To show how malicious scripts can be injected into web pages and executed in users' browsers.
- **Broken Authentication or Session Management:** To explore how weak login mechanisms or session handling can lead to unauthorized access.

For each vulnerability, we documented the testing steps, captured proof of exploitation, explained the root cause, and recommended secure coding fixes to prevent future attacks.

Tools Used

- Operating system: Kali linux
- Platform Used: Docker
- Targeted Application: OWASP Juice Shop
- Command line Interface: Terminal

Task 1: Understanding OWASP Top 10 Vulnerability

- a. The OWASP Top 10 is a regularly update list of the 10 most critical security risks to web applications. It's based on data from real-world vulnerabilities reported globally

Why is the OWASP Top 10 important?

- i. It help developers, testers, and organizations understand common security threats.
 - ii. Many security standards and regulations (like ISO, PCI-DSS) reference the OWASP Top 10.
 - iii. Following it reduces the likelihood of common, high-impact vulnerabilities (e.g, SQL Injection, Cross-site Scripting).
 - iv. It guides secure coding practices and helps integrate security early in the pocess.
 - v. It's Often used as a baseline for security audits and compliance requirements.
- b. **A01:2025-Broken Access Control:** Allows users to access data or perform actions they are not authorized for.

Real-world breach: Facebook (2019) — A bug allowed users to view and delete private posts via insecure direct object references (IDOR).

A02:2025-Security Misconfiguration: Occurs when security settings are not properly configured, such as enabling debugging features in production or leaving default credentials unchanged.

Real-World Breach: Capital One (2019) — Misconfigured AWS firewall allowed access to over 100 million customer accounts.

A03:2025-Software and Data Integrity Failures: Deals with failure in Protecting against integrity Violations due to insecure or outdated software and data

Real-World Breach: Marriot/Starwood Breach (2018) — Attackers accessed and exfiltrated data for 4 years before detection.

A04:2025-Cryptographic Failures: Refers to vulnerabilities in the encryption of data at rest and in transit, including the use of weak algorithms or improper key management.

Real-World Breach: Equifax (2017) — Poor encryption and expired SSL certificates led to a breach exposing 147 million users' data.

A05:2025- Injection: Occurs when an attacker can send untrusted data to an interpreter, such as a database or operating system.

Real-World Breach: Heartland Payment Systems (2008) — SQL Injection allowed attackers to steal 100M+ credit card records.

A06:2025-Insecure Design: Highlights risks related to design and architectural flaws rather than implementation errors.

Real-World Breach: Zoom (2020) — Weak default security settings and poor design enabled "Zoombombing" attacks.

A07:2025-Authentication Failures: Addressing issues where user identities are not properly verified or managed, leading to unauthorized access.

Real-World Breach: Uber Data Breach (2016)-AWS credentials were left in a public GitHub repo, enabling attacker access to customer data.

A08:2025-Software Supply Chain Failures: Occurs when logging, monitoring, and alerting are not adequate, making it difficult to detect and respond to attacks.

Real-World Breach: Event-Stream NPM Package (2018) — malicious code injected into a popular library used in many apps.

A09:2025-Logging & Alerting Failures: Occurs when logging, monitoring, and alerting are not adequate, making it difficult to detect and respond to attacks.

Real-World Breach: Target (2013) — Failure to act on security alerts resulted in a delay in detecting a breach affecting 40 million cards.

A10:2025-Server-Side Request Forgery (SSRF): Mishandling of exceptional conditions allows an attacker to cause the server to make unintended or malicious requests to internal or external resources.

Real-World Breach: Capital One (2019) — SSRF vulnerability exploited to access AWS metadata service.

Task 2: Application Risk Mapping

1. A01: Broken Access Control

- **How it could exist:** PayX Secure may lack proper authorization checks on the endpoints like transfer funds or view transactions, allowing users to access or manipulate data not to them.
- **Exploitation:** An attacker could modify request parameters (e.g. user IDs) to access or transfer funds from other user's accounts using insecure Direct Object References (IDOR).
- **Business Impact:** Unauthorized fund transfers, legal liabilities, loss of customer trust, and regulatory penalties (e.g. GDPR, PCI DSS violations).

2. A02: Security Misconfiguration

- **How it could exist:** Default admin credentials, verbose error messages, or misconfigured cloud storage (e.g. open S3 buckets) could expose sensitive data or system internals.
- **Exploitation:** Attackers could scan for misconfigured services, access debug endpoints, or exploit overly permissive cloud settings to exfiltrate data or escalate privileges.
- **Business Impact:** Data breaches, reputational damage, and increased remediation costs due to exposed infrastructure.

3. A07: Identification and Authentication Failures

How it could exist: Weak password policies, lack of multi-factor authentication (MFA), or improper session management could allow attackers to hijack accounts.

Exploitation: Credential stuffing or brute-force attacks could allow unauthorized access to user wallets. Session fixation or token reuse could further compromise accounts.

Business Impact: Account takeovers, fraudulent transactions, customer churn, and potential financial losses.

4. A08: Software and Data Integrity Failures

How it could exist: PayX Secure might rely on third-party libraries or CI/CD pipelines without validating the integrity of software updates or dependencies.

Exploitation: Attackers could inject malicious code into a third-party library or compromise the build pipeline, leading to widespread malware distribution.

Business Impact: System compromise, data theft, regulatory scrutiny, and loss of platform credibility.

5. A09: Logging and Monitoring Failures

How it could exist: Inadequate logging of critical events like failed logins, fund transfers, or API misuse could delay detection of attacks.

Exploitation: Attackers could perform stealthy operations (e.g. data scraping, brute-force attacks) without triggering alerts or being noticed.

Business Impact: Prolonged breaches, increased damage, delayed incident response, and higher recovery costs.

Task 3: Practical Vulnerability Testing

Installation and Execution Report on OWASP Juice Shop using Docker.

To update the package index I use `sudo apt update`

```
(raph@kali)-[~]
└─$ sudo apt update
[sudo] password for raph:
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [20.9 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.7 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [114 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [259 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [188 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [894 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [11.7 kB]
Get:9 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [28.5 kB]
Fetched 75.2 MB in 1min 14s (1,016 kB/s)
1265 packages can be upgraded. Run 'apt list --upgradable' to see them.
```

Installing Docker I use `sudo apt install docker.io`

```
(raph@kali)-[~]
└─$ sudo apt install docker.io
The following packages were automatically installed and are no longer required:
amass-common libbluray2 libtotem0 python3-pyfiglet
binutils-mingw-w64-i686 libbson-1.0-0t64 libudfread0 python3-pyshodan
binutils-mingw-w64-x86_64 libdmapsharing-4.0-3t64 libx264-164 python3-qasync
dnsmasq libgom-1.0-0t64 libxml2 python3-qrcode
dsniff libgrilo-0.3-0 libyelp0 python3-serial-asyncio
ettercap-common libjs-jquery-ui medusa python3-smmmap
ettercap-graphical liblua5.2-0 mingw-w64-common python3-tld
figlet liblua5.1-2 mingw-w64-i686-dev python3-yaswfp
finger liblua5.1-common mingw-w64-x86_64-dev python3-zombie-imp
gcc-mingw-w64-base libmongocrypt0 oracle-instantclient-basic rsh-redone-client
gcc-mingw-w64-i686-win32 libmongocrypt0 python3-bluepy samba-ad-dc
gcc-mingw-w64-i686-win32-runtime libnids1.21t64 python3-click-plugins samba-ad-provision
gcc-mingw-w64-x86_64-win32 liboauth0 python3-git samba-dsdb-modules
gcc-mingw-w64-x86_64-win32-runtime libpeas-1.0-0 python3-gitdb smtp-user-enum
girl1.2-peas-1.0 libpeas-common python3-gpg sparta-scripts
girl1.2-totem-1.0 libplacebo349 python3-kismetcapturebtgeiger toilet-fonts
girl1.2-totemplparser-1.0 libportal-gtk3-1 python3-kismetcapturefreaklabszigbee totem
grilo-plugins-0.3 libportmidi0 python3-kismetcapturertl433 totem-common
imagemagick libbravie0.7 python3-kismetcapturertladsb totem-plugins
imagemagick-7.q16 libsmb2-6 python3-kismetcapturertlamr unicornsans
libaio1t64 libtheoraec1 python3-pefile urlscan
libapache2-mod-php libtheoraenc1 python3-pyexploitdb wapiti
Use 'sudo apt autoremove' to remove them.
Installing:
```

This downloads the latest image of Juice Shop from dockerhub

```
(raph@kali)-[~]
$ sudo docker pull bkimminich/juice-shop
Using default tag: latest
latest: Pulling from bkimminich/juice-shop
fd4aa3667332: Pull complete
bfb59b82a9b6: Pull complete
017886f7e176: Pull complete
62de241dac5f: Pull complete
2780920e5dbf: Pull complete
7c12895b777b: Pull complete
3214acf345c0: Pull complete
5664b15f108b: Pull complete
045fc1c20da8: Pull complete
4aa0ea1413d3: Pull complete
da7816fa955e: Pull complete
ddf74a63f7d8: Pull complete
e7fa9df358f0: Pull complete
d8a0d911b13e: Pull complete
5b14f6c9a813: Pull complete
33ce0b1d99fc: Pull complete
f45e0372ce60: Pull complete
7faf0cfa885c: Pull complete
9cd2a1476fcc: Pull complete
7b72e6384ef9: Pull complete
0168f69dfb16: Pull complete
Digest: sha256:1c55debeaf4fd5678019b17818a539e1e06ef93d29b268a21f53f0773a9fff5d
Status: Downloaded newer image for bkimminich/juice-shop:latest
```

This command logged me in to docker

```
(raph@kali)-[~]
$ sudo docker login

USING WEB-BASED LOGIN
To sign in with credentials on the command line, use 'docker login -u <username>'

Your one-time device confirmation code is: RKCF-QPCP
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser...

WARNING! Your password will be stored unencrypted in /root/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credential-stores

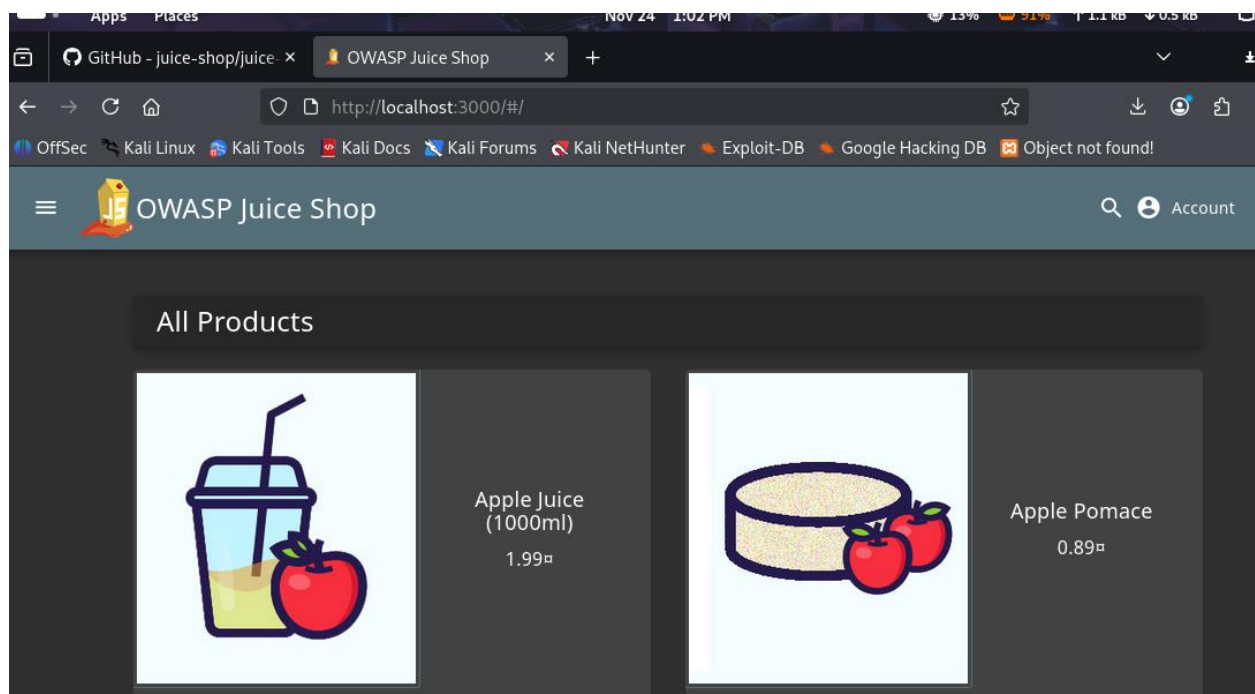
Login Succeeded
```

This command launches the Juice Shop app and maps it to the port 3000

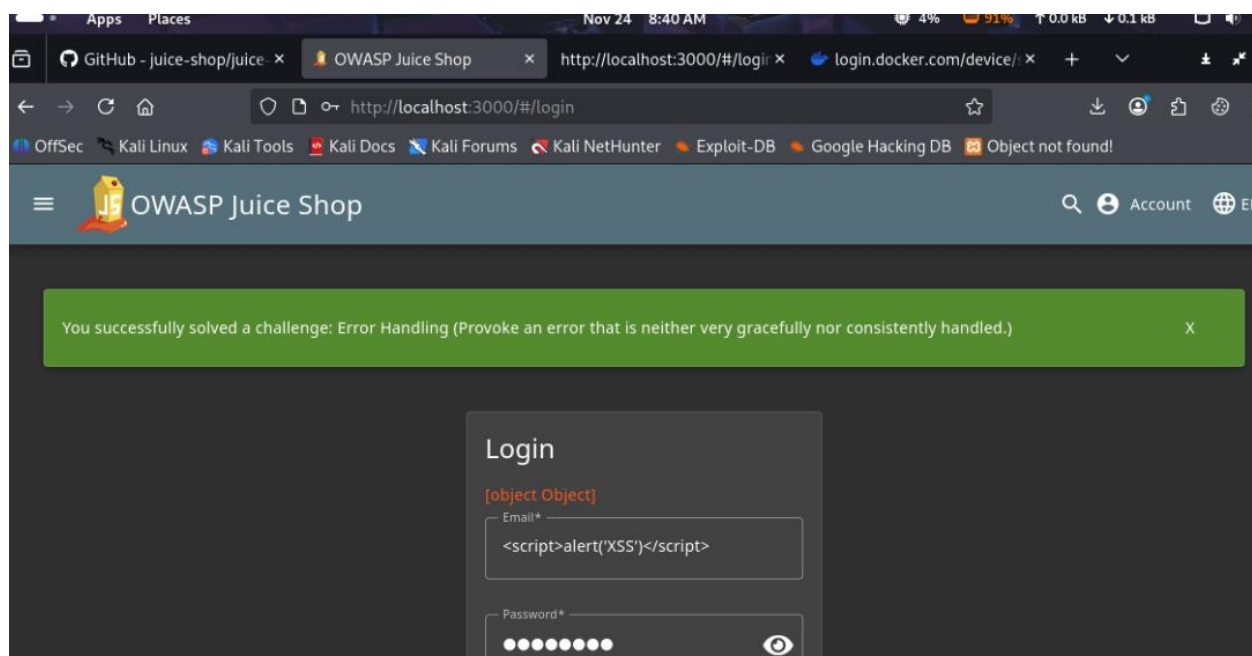
```
(raph@kali)-[~]  
$ sudo docker run -d -p 3000:3000 bkimminich/juice-shop  
61ed8aa7fde3d1ab712408e95a44c3e91c06a882e6a6ace8e910009196964e51
```

Opened the web browser and navigated to <http://localhost:3000>.

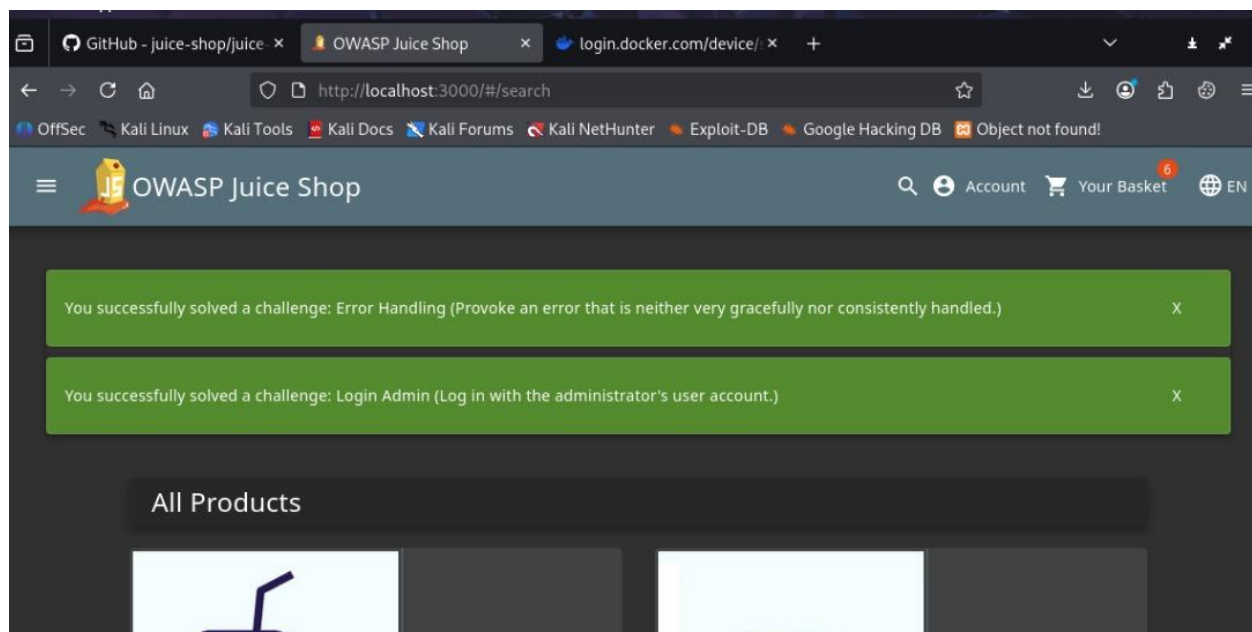
OWASP Juice Shop is now successfully installed and running via Docker my kali linux machine. The platform is ready for testing vulnerabilities such as SQL injection, Cross-site Scripting (XSS), and Broken Authentication in a safe environment.



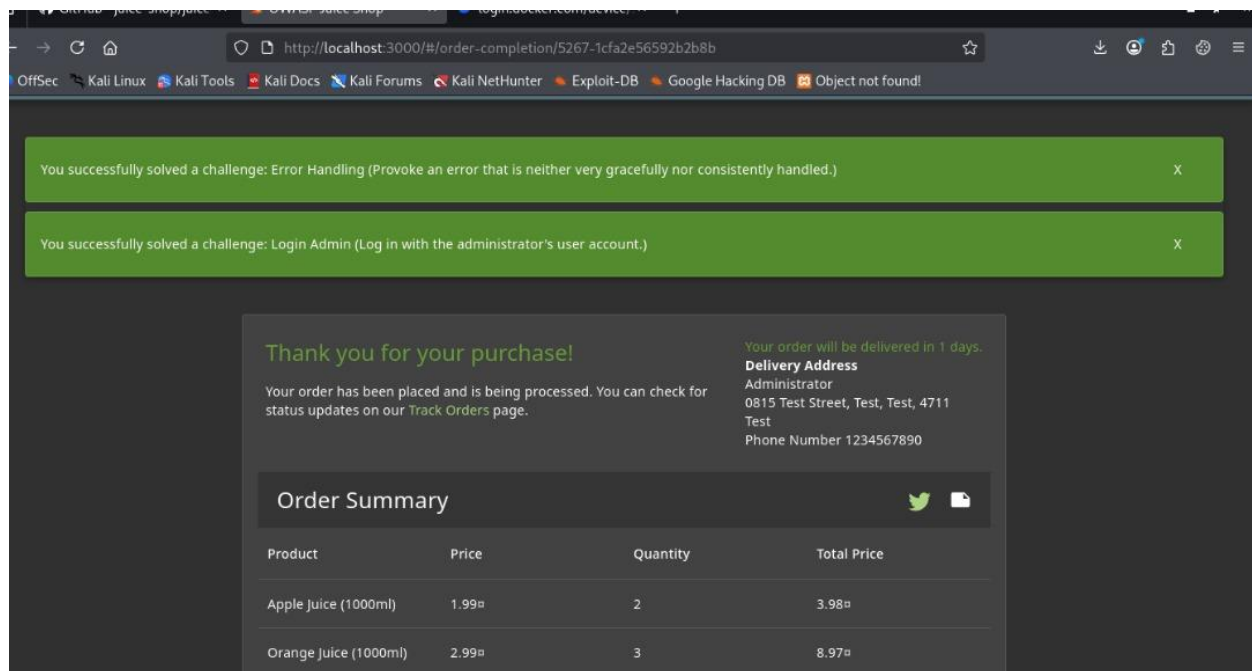
This alert('XSS') payload caused the app to improperly handle the input, displaying [object Object] and exposing poor error management. This is a great example of how improper input sanitization and ungraceful error messages can be exploited.



In the email/username input, I tested common SQLi payloads such as admin' – and ' OR 1=1-- . Submitted the form with any password. Gained access to the admin account, triggering the “Login Admin” challenge success.



After the successful login, I ordered some products in the juice shop and my order was successful.



The screenshot shows a web browser window with the URL `http://localhost:3000/#/order-completion/5267-1cfa2e56592b2b8b`. The browser's address bar and tabs are visible at the top. Below the browser window, the page content is displayed on a dark background.

Two green success messages are shown at the top:

- You successfully solved a challenge: Error Handling (Provoke an error that is neither very gracefully nor consistently handled.)
- You successfully solved a challenge: Login Admin (Log in with the administrator's user account.)

Below these messages, a section titled "Thank you for your purchase!" is displayed. It includes the text: "Your order has been placed and is being processed. You can check for status updates on our [Track Orders](#) page."

To the right of this text, the delivery address is listed:

Your order will be delivered in 1 days.
Delivery Address
Administrator
0815 Test Street, Test, Test, 4711
Test
Phone Number 1234567890

Below the thank you message, an "Order Summary" section is shown. It includes a table with the following data:

Product	Price	Quantity	Total Price
Apple Juice (1000ml)	1.99	2	3.98
Orange Juice (1000ml)	2.99	3	8.97

Vulnerability Tested: Broken Authentication / Session Management

Test Performed:

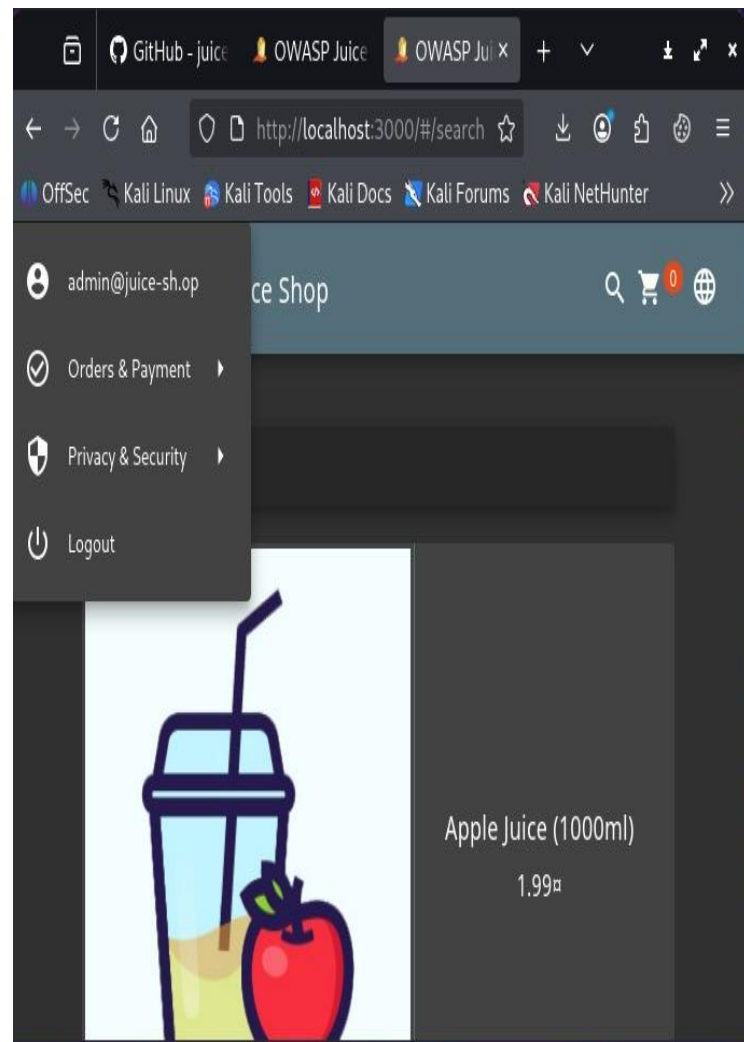
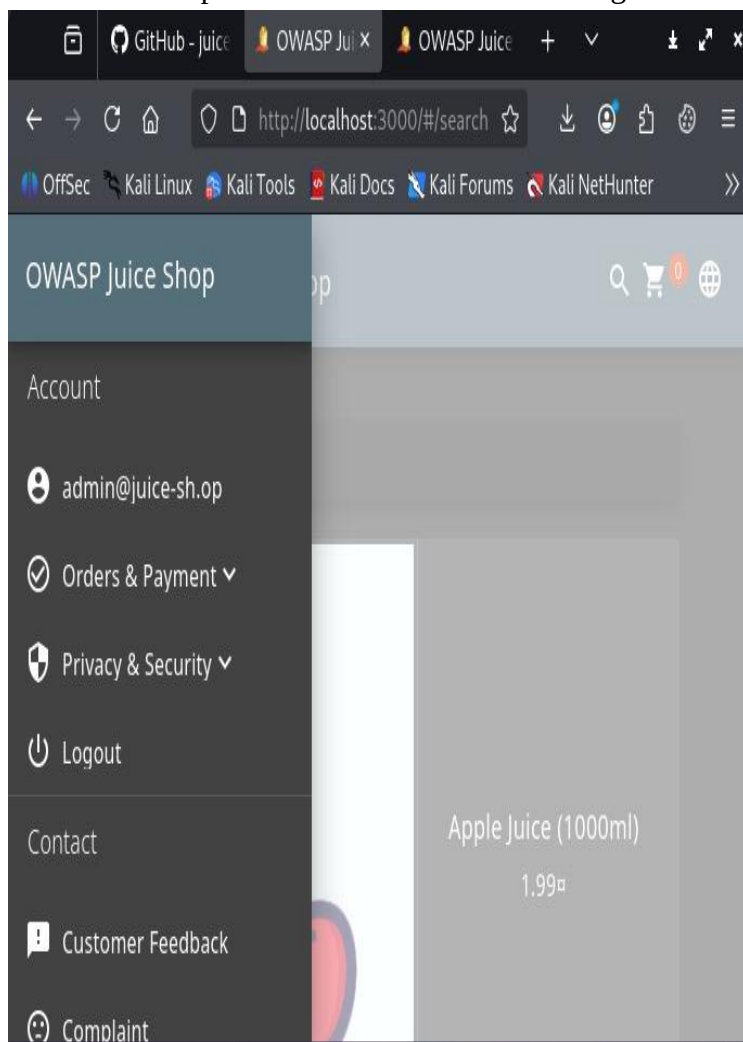
- Logged into OWASP Juice Shop using valid credentials.
- Opened Developer Tools > Application > Cookies to copy the session token (JWT).
- Pasted the token into a new browser tab/session and accessed the account — it successfully logged in.
- Logged out from the original tab.
- The second session was also terminated — user was logged out.

Result:

Session was invalidated on logout from the original session. This shows Juice Shop properly invalidates session tokens on logout, preventing session hijacking or reuse.

Conclusion:

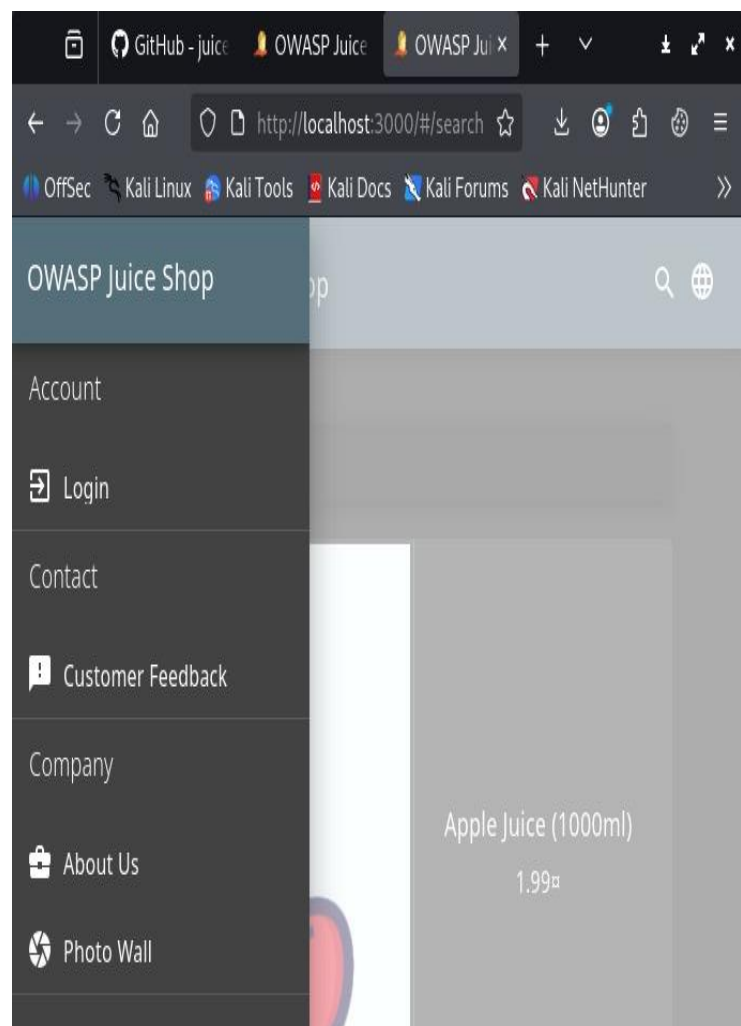
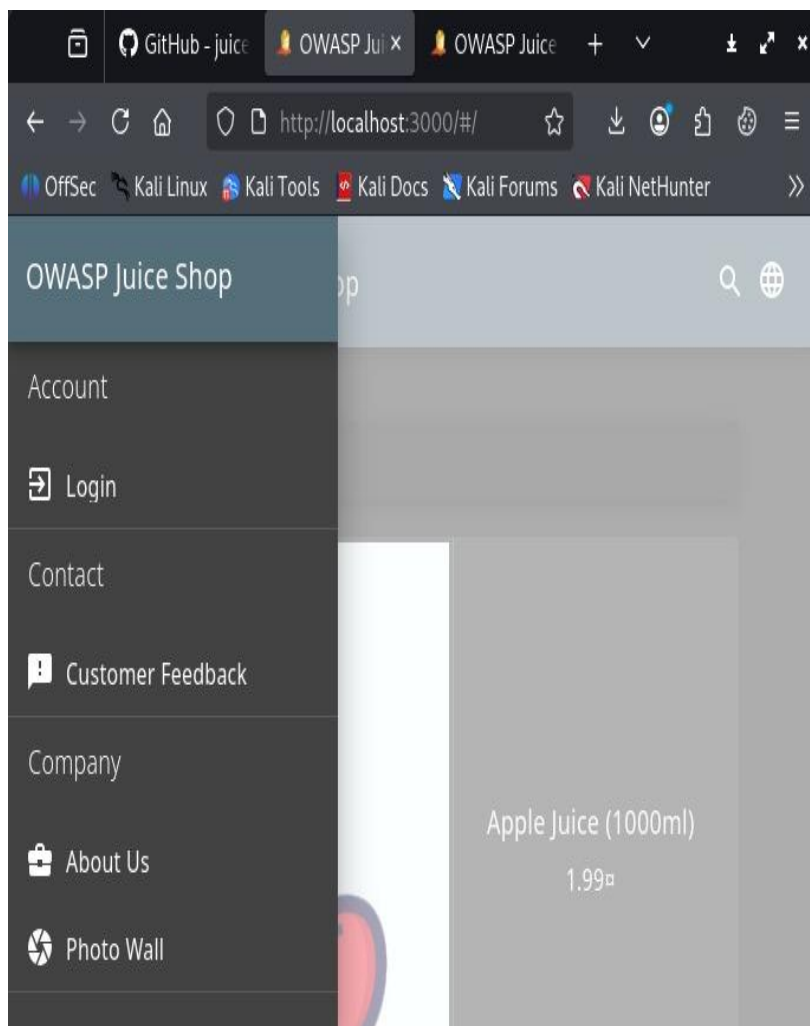
No broken session management vulnerability found in this test. The application correctly implements secure session handling.



This screenshot captures the behavior of the application after logging out from the main tab. The copied session in the second tab was also invalidated automatically.

What it proves:

- Session tokens are properly invalidated upon logout.
- Logging out from one tab ends the session across all instances where the same token was used.
- This indicates secure session handling and no Broken Authentication or Session Management vulnerability in this case.



Secure Fix Recommendation

1. XSS (Cross-Site Scripting):

- Always sanitize and escape user input. Use libraries or frameworks that auto-escape (like React or Angular).
- Implement Content Security Policy (CSP) headers to limit scripts.
- Validate input on both client and server side.

2. SQLi (SQL Injection):

- Use prepared statements (parameterized queries) with libraries like PDO (PHP), or ORM frameworks like Sequelize, SQLAlchemy, etc.
- Avoid concatenating user input in SQL queries.
- Enforce least privilege on the database account.

3. Session Management:

- Use secure, HttpOnly cookies for session tokens.
- Invalidate sessions on logout or after inactivity.
- Use multi-factor authentication (MFA) and random, long session IDs.
- Avoid exposing session tokens in URLs.

Task 4: Secure Design Recommendations

Risk	Your Recommendation
SQL Injection	Implement input validation and sanitization using whitelists.
Broken Access Control	Validate access permissions server-side, never rely on client-side enforcement.
Security Misconfiguration	Disable debugging and verbose error messages in production.

Vulnerable Components	Subscribe to security advisories and update components promptly.
SSRF	Validate and sanitize all user-supplied URLs.

Reference

- Bargestan, N. (2023). Real-world breaches mapped to OWASP Top 10. Medium. <https://medium.com/@nimabargestan/real-world-breaches-mapped-to-owasp-top-10-fcda69bcbe94>
- YouTube: www.youtube.com/watch?v=QCxhywZecq8&utm_source=chatgpt.com
- OWASP. (n.d.). OWASP Juice Shop [Source code]. GitHub. <https://github.com/juice-shop/juice-shop>